ПУТЕВОДИТЕЛЬ ПО КОДУ ПРИЛОЖЕНИЯ «МАРШРУТ»

Здесь описано, где что лежит в репозитории и за что отвечает каждая папка и каждый файл. Сам код подробно прокомментирован по-русски: в начале каждого файла сказано, за что он отвечает, у каждого класса и метода есть описание, а внутри методов пояснены шаги. Путеводитель помогает найти нужный файл, а комментарии в нём объясняют, как он работает.

1. С чего начать чтение

1. `lib/main.dart` — точка входа: открывает настройки и запускает приложение. Своей базы у приложения нет.
2. `lib/app.dart` — корневой виджет: тема оформления, русский язык, роутер.
3. `lib/core/router/app_router.dart` — все экраны и правило, кого куда пускать без входа.
4. `lib/domain/planner/` — главный алгоритм: как из тем, теста и расписания получается план.
5. `lib/presentation/screens/` — экраны, по одной папке на раздел приложения.
6. `lib/data/api/api_client.dart` — как приложение отправляет запросы серверу и обрабатывает ответы и ошибки.
7. `lib/server/api_server.dart` — сервер API: все адреса, проверка токена и данных, коды ответов.

2. Корень репозитория

Таблица 1 — Папки и файлы в корне репозитория

Папка или файл	Что внутри
`lib/`	Весь код на Dart: приложение и сервер API (подробно в разделе 4)
`bin/marshrut_server.dart`	Программа-сервер API: разбирает параметры (`--port`, `--db`, `--catalog`), открывает базу, читает каталог и запускает сервер; печатает адреса для телефона

Папка или файл	Что внутри
`assets/catalog/`	Каталог тем: по одному JSON-файлу на направление (`flutter.json`, `python.json`, `english.json`, `algorithms.json`) с темами, связями между ними и вопросами теста
`test/`	Модульные и виджет-тесты, в том числе сервера и клиента API (раздел 5)
`integration_test/`	Интеграционный тест настоящего приложения с настоящим HTTP-сервером API
`tool/`	Вспомогательные программы: снимки экранов, иконки, сборка документов курсовой, файл запуска сервера (раздел 6)
`docs/`	Курсовая работа, концепция, критерии, памятки к защите
`android/`, `ios/`, `windows/`	Настройки сборки под Android, iPhone и Windows: название, иконка, разрешения на уведомления и на работу с сервером в локальной сети по HTTP
`linux/`, `web/`	Заготовки Flutter для других платформ; Linux нужен для интеграционного теста в CI
`.github/workflows/build.yml`	GitHub Actions: проверка форматирования, анализ, тесты, сборка APK для Android, IPA для iPhone, версии для Windows и программы-сервера; файлы для релиза
`.gitattributes`	Файлы `.bat` хранятся с окончаниями строк Windows, иначе «Запустить сервер.bat» не запустится
`pubspec.yaml`	Название, версия и зависимости приложения и сервера (Riverpod, go_router, http, shelf, shelf_router, drift, fl_chart, flutter_local_notifications и др.)
`analysis_options.yaml`	Правила статического анализа (`flutter analyze`)
`README.md`	Описание проекта, как запустить, где скачать сборку

3. Как связаны части: путь одного действия

Система — это две программы: приложение и сервер API. Код приложения разделён на три слоя. **presentation** (экраны и провайдеры) вызывает **domain** (сценарии и алгоритм), а domain работает с данными через интерфейсы, которые реализует слой **data**. В приложении эти интерфейсы реализуют репозитории API (`lib/data/api/`): каждый метод отправляет запрос серверу. Сервер (`lib/server/api_server.dart`) проверяет запрос и работает с базой SQLite через те же интерфейсы, но реализованные репозиториями drift (`lib/data/repositories/`). Провайдеры Riverpod в `lib/presentation/providers/providers.dart` создают все объекты приложения и передают их экранам.

Пример: пользователь на шаге «Расписание» нажимает «Сформировать план».

1. screens/onboarding/schedule_screen.dart открывает предпросмотр (/onboarding/preview). Выбранные направление, ответы теста и расписание хранит providers/onboarding_controller.dart.
2. screens/onboarding/preview_screen.dart вызывает PlanService.preview() из lib/domain/usecases/plan_service.dart.
3. PlanService передаёт данные в PlanGenerator.generate() (lib/domain/planner/plan_generator.dart): исключение известных тем (knowledge_estimator.dart), порядок тем (topological_sort.dart), раскладка по дням (scheduler.dart), проверка дедлайна.
4. По кнопке «Начать обучение» вызывается PlanService.createPlan(), который сохраняет план через интерфейс PlanRepository. В приложении это ApiPlanRepository (lib/data/api/api_plan_repository.dart): он переводит план в JSON (api_json.dart) и через ApiClient отправляет запрос POST /api/plans с токеном входа.
5. Сервер (lib/server/api_server.dart, обработчик _createPlan) проверяет токен и данные и вызывает PlanRepositoryImpl.createPlan() (lib/data/repositories/plan_repository_impl.dart), который записывает план, темы и занятия в базу lib/data/db/app_database.dart. В ответ приходит код 201 и номер плана, а в окне сервера появляется строка журнала.
6. ApiClient сообщает, что данные изменились, и главный экран (screens/home/home_screen.dart), подписанный на активный план (activePlanProvider), загружает его заново (GET /api/plans/active) и показывает занятия на сегодня.

Если сервер не ответил, `ApiClient` выбрасывает `ConnectionException` с понятным текстом, и экран показывает его пользователю; данные при этом не меняются.

Так же устроены остальные действия: отметка занятия — `PlanService.toggleSession()`, кнопка «Перестроить» — `PlanService.replan()` и `lib/domain/planner/replanner.dart`, вход — `AuthController`, `ApiAuthRepository` и на сервере `AuthRepositoryImpl`.

4. Папка lib/ — код приложения

Ниже каждая папка `lib/` и каждый файл в ней. После слова «Главное» перечислены классы и функции, с которых стоит начинать чтение файла.

4.1 Корень `lib/`

Таблица 2 — Точка входа и корневой виджет (папка lib/)

Файл	**За что отвечает**
`main.dart`	Точка входа: инициализирует Flutter и русские даты, открывает SharedPreferences, подставляет их в провайдеры и запускает приложение. Своей базы у приложения нет: данные оно получает через сервер API. Автоматический повтор упавших запросов Riverpod отключён (`noAutoRetry`), чтобы при выключенном сервере ошибка показывалась сразу. Главное: main(), ProviderScope (retry: noAutoRetry, overrides: sharedPreferencesProvider).
`app.dart`	Корневой виджет приложения: подключает светлую и тёмную темы, выбранный пользователем режим темы, роутер и русскую локализацию. Главное: MarshrutApp, build() → MaterialApp.router.

4.2 `lib/core/` — общее для всего приложения

Таблица 3 — Роутер, тема оформления и утилиты (папка lib/core/)

Файл	За что отвечает
`router/app_router.dart`	Карта всех экранов (go_router) и правило доступа: без входа — только вход и регистрация, пока сессия проверяется — заставка. Главное: authRedirect(), routerProvider, publicRoutes, StatefulShellRoute с 5 вкладками (Главная, План, Темы, Статистика, Профиль).
`theme/app_theme.dart`	Фирменные цвета (тёмно-синий и оранжевый) и темы Material 3 со стилями шапки, карточек, полей, кнопок и навигации. Главное: AppColors, AppTheme.light(), AppTheme.dark(), _build().
`utils/formatters.dart`	Перевод дат, минут и часов в русский текст и склонение слов после чисел. Главное: Fmt (day(), dayWithWeekday(), minutes(), hours(), hoursWord(), plural(), count()), weekdayShort.
`utils/validators.dart`	Проверки полей форм (имя, email, пароль, повтор пароля, название, часы), которые передаются в TextFormField.validator. Главное: Validators (name(), email(), password(), loginPassword(), confirmPassword(), title(), hours(), parseHours()).
`utils/password_hasher.dart`	Безопасное хранение паролей: PBKDF2-HMAC-SHA256 со случайной солью, расчёт в фоновом изоляте, проверка старых хэшей SHA-256 и их замена. Главное: PasswordHasher (generateSalt(), hash(), hashAsync(), verify(), verifyAsync(), needsRehash(), pbkdf2(), _constantTimeEquals()).

4.3 `lib/domain/entities/` — модели данных

Сущности — это обычные классы Dart без Flutter и без базы данных: тема, направление, план, занятие, пользователь, заметка. Ими обмениваются все слои.

Таблица 4 — Сущности предметной области (папка lib/domain/entities/)

Файл	**За что отвечает**
`user.dart`	Модель пользователя: имя, email, цвет аватара, настройки ежедневного напоминания. Главное: AppUser, геттер initials.
`direction.dart`	Направление обучения (граф тем), загружаемое из assets/catalog/*.json. Главное: Direction, Direction.fromJson(), topicById(), diagnosticTopics, totalMinutes.
`topic.dart`	Тема (вершина графа) с длительностью, уровнем, зависимостями, материалами и вопросом диагностического теста. Главное: Topic, Topic.fromJson(), Resource, DiagnosticQuestion.
`schedule_settings.dart`	Настройки расписания: часы в неделю, учебные дни, дата начала, дедлайн и расчёт лимита минут на день. Главное: ScheduleSettings, dailyLimitMinutes, copyWith().
`session.dart`	Учебное занятие: рассчитанное планировщиком и сохранённое в базе, тип «изучение/повторение», признак пропуска. Главное: SessionKind, PlannedSession, StudySession (isMissed(), displayTitle).
`plan.dart`	Модели плана: результат генератора, сохранённый план, снимок темы в плане и сводка прогресса. Главное: PlanResult, StudyPlan, PlanTopic.toTopic(), PlanProgress.fromSessions().
`note.dart`	Модели заметки, завершённого интервала Pomodoro и записи истории активности. Главное: Note, PomodoroEntry, HistoryEntry, HistoryKind.

4.4 `lib/domain/planner/` — алгоритм построения плана

Главная часть курсового проекта. Читать удобно в таком порядке: `plan_generator.dart` (общая схема из пяти шагов), затем каждый шаг по отдельности.

Таблица 5 — Алгоритм построения плана (папка lib/domain/planner/)

Файл	За что отвечает
`plan_generator.dart`	Генератор плана из 5 шагов: исключить известные темы → сортировка Кана → раскладка по дням → проверка дедлайна. Если план не укладывается в дедлайн, перебором с шагом 0,5 ч подбирает нужное число часов в неделю. Главное: PlanGenerator, generate(), requiredHoursPerWeek(), lastStudyDate().
`knowledge_estimator.dart`	По ответам диагностического теста определяет, какие темы пользователь уже знает. Верный ответ отмечает тему и всех её предшественников (обход графа через стек), неверный ответ снимает отметку с темы. Главное: KnowledgeEstimator, knownTopics().
`topological_sort.dart`	Топологическая сортировка тем алгоритмом Кана: входящие степени, очередь с приоритетом по уровню и исходной позиции, обнаружение циклов. Главное: topologicalSort(), CycleDetectedException.
`scheduler.dart`	Раскладывает упорядоченные темы по календарю день за днём. В каждый учебный день действует лимит минут; сначала ставятся повторения, срок которых наступил, затем куски текущей темы (не короче 30 минут). После завершения темы планируются повторения через 1, 3 и 7 дней. Главное: Scheduler, schedule(), PendingReview, _numberParts().
`replanner.dart`	Перестраивает план с сегодняшнего дня. Выполненные занятия остаются, остаток минут по темам раскладывается заново, просроченные повторения переносятся на сегодня, номера частей продолжают уже пройденные. Главное: Replanner, replan(), ReplanResult.
`dates.dart`	Функции для работы с датами без учёта времени суток, устойчивые к переходу на летнее время; ими пользуются планировщик, сервисы и экраны. Главное: dateOnly(), addDays(), daysBetween(), isSameDay(), countStudyDays().

4.5 `lib/domain/usecases/` и `lib/domain/repositories/` — сценарии и интерфейсы

Интерфейс говорит, что умеет хранилище (например, «создать план»), но не как. У каждого интерфейса данных две реализации: в приложении — через API (`lib/data/api/`), на сервере — через базу (`lib/data/repositories/`). Поэтому алгоритм и сценарии не зависят от того, где лежат данные, и легко проверяются тестами.

Таблица 6 — Сценарии и интерфейсы хранилищ (папка lib/domain/)

Файл	За что отвечает
`usecases/plan_service.dart`	Сценарии работы с планом, связывает алгоритмы с хранилищем PlanRepository: предпросмотр, создание, отметка занятия, перепланирование, смена расписания, добавление и удаление темы. Главное: PlanService, createPlan(), replan(), addTopic(), removeTopic().
`repositories/repositories.dart`	Интерфейсы хранилищ слоя domain («что нужно приложению») и общая ошибка для пользователя; реализации лежат в lib/data. Главное: AppException (и ConnectionException — нет связи с сервером), CatalogRepository, AuthRepository, PlanRepository, TopicRepository, NoteRepository, PomodoroRepository, SessionStorage, ReminderService.

4.6 `lib/data/` — данные: клиент API, база сервера, уведомления

Приложение использует папку `api/`: клиент API и репозитории, которые обращаются к серверу. Папки `db/`, `repositories/` (кроме `session_storage_impl.dart`) и `datasources/` использует сервер. Файл `db/app_database.g.dart` создан генератором drift (`dart run build_runner build`) по описанию таблиц из `app_database.dart`. Его не правят вручную, поэтому он не комментировался.

Таблица 7 — Файлы слоя данных (папка lib/data/)

Файл	За что отвечает
`api/api_client.dart`	Клиент HTTP: через него проходят все запросы приложения к данным. Приводит адрес сервера к виду `http://хост:порт`, добавляет токен в заголовок `Authorization: Bearer`, переводит тело в JSON, ждёт ответ не дольше 10 секунд и превращает любую неудачу в понятное сообщение: нет связи, сервер не отвечает, ошибка 4xx/5xx с текстом от сервера, ответ не JSON. При ответе 401 вызывает onUnauthorized. Метод watch делает «самообновляющийся» поток: загружает данные заново после каждого изменения и повторяет запрос через 5 секунд после ошибки. Главное: ApiClient (get(), post(), put(), patch(), delete(), _send(), checkHealth(), watch(), normalizeUrl(), changes), ApiException.
`api/api_json.dart`	Перевод сущностей (пользователь, план, тема, занятие, заметка) в JSON и обратно. Им пользуются и сервер, и приложение, поэтому формат данных описан в одном месте. Главное: Json, userToJson()/userFromJson(), planToJson()/planFromJson(), studySessionToJson()/studySessionFromJson(), plannedSessionToJson(), noteToJson()/noteFromJson(), dateToJson().

Файл	За что отвечает
api/api_auth_repository.dart	Регистрация, вход, профиль, смена пароля, напоминание и удаление аккаунта запросами /api/auth/... и /api/me. После входа запоминает токен в клиенте API. Пароли приложение не хэширует и не хранит: их проверяет сервер. Главное: ApiAuthRepository (register(), login(), logout(), userById(), updateProfile(), changePassword(), deleteAccount()).
api/api_catalog_repository.dart	Каталог направлений с сервера (GET /api/directions): загружается один раз и дальше берётся из памяти. Главное: ApiCatalogRepository, loadDirections().
api/api_plan_repository.dart	Планы, темы планов и занятия через /api/plans/... и /api/sessions/.... План составляет алгоритм в приложении, а сервер его сохраняет и отдаёт. Главное: ApiPlanRepository (createPlan(), watchPlans(), watchActivePlan(), setActive(), renamePlan(), deletePlan(), setSessionDone(), replaceOpenSessions()).
api/api_topic_repository.dart	Свои темы (полный CRUD через /api/custom-topics) и избранное (/api/favorites). Главное: ApiTopicRepository (watchCustomTopics(), createCustomTopic(), updateCustomTopic(), deleteCustomTopic(), toggleFavorite()).
api/api_note_repository.dart	Заметки — главный пример полного CRUD через API: GET, POST, PUT, DELETE по адресам /api/notes; журнал Pomodoro — /api/pomodoros. Главное: ApiNoteRepository (watchNotes(), createNote(), updateNote(), deleteNote()), ApiPomodoroRepository (logPomodoro(), watchPomodoros()).
db/app_database.dart	Схема базы SQLite на сервере (drift): 9 таблиц, внешние ключи с каскадным удалением, включение foreign_keys при открытии базы, переход со схемы версии 1 на версию 2 (добавлена таблица токенов). Код запросов генерируется в app_database.g.dart. Главное: AppDatabase, Users, Plans, PlanTopics, Sessions, CustomTopics, Notes, Favorites, PomodoroLogs, ApiTokens, migration.
datasources/catalog_repository_impl.dart	Загружает каталог направлений (Flutter, Python, английский, алгоритмы) из JSON-файлов и кеширует результат. Если JSON повреждён, выдаёт понятную ошибку. Список файлов каталога берёт сервер API. Главное: CatalogRepositoryImpl, files, loadDirections(), AssetReader.
repositories/auth_repository_impl.dart	Регистрация и вход на сервере: пользователи лежат в таблице users, пароли хранятся только как хэш с солью, хэши старого формата пересчитываются при входе. Также профиль, смена пароля, удаление аккаунта и настройки напоминания. Главное: AuthRepositoryImpl, register(), login(), updateProfile(), changePassword(), deleteAccount(), updateReminder(), avatarPalette.
repositories/session_storage_impl.dart	Используется приложением: хранит в shared_preferences адрес сервера API (по умолчанию http://localhost:8080), id вошедшего пользователя и его токен («Запомнить меня») и выбранную тему оформления. Реализует интерфейс SessionStorage. Главное: SessionStorageImpl, defaultServerUrl, serverUrl, currentUserId, authToken, saveSession(), themeMode.

Файл	За что отвечает
repositories/plan_repository_impl.dart	На сервере хранит учебные планы, их темы и занятия (таблицы plans, plan_topics, sessions). Активным может быть только один план; есть транзакции при создании и удалении плана и при перепланировании. Главное: PlanRepositoryImpl, createPlan(), setActive(), deletePlan(), replacePlanTopics(), replaceOpenSessions(), watchCompletedSessions().
repositories/topic_repository_impl.dart	На сервере хранит свои темы пользователя с id вида custom_<n> и избранное (таблицы custom_topics и favorites). Темы каталога через него менять нельзя. Главное: TopicRepositoryImpl, watchCustomTopics(), createCustomTopic(), updateCustomTopic(), deleteCustomTopic(), toggleFavorite(), directionOf().
repositories/note_repository_impl.dart	На сервере хранит заметки пользователя (таблица notes) и журнал завершённых «помидоров» (таблица pomodoro_logs) в SQLite через drift. Главное: NoteRepositoryImpl (watchNotes(), createNote(), updateNote(), deleteNote()), PomodoroRepositoryImpl (logPomodoro(), watchPomodoros()).
services/reminder_service_impl.dart	Уведомления через flutter_local_notifications: ежедневное напоминание об учёбе (только Android и iOS) и мгновенные уведомления (ещё и Windows). Плюс заглушка для тестов. Главное: LocalReminderService (init(), requestPermission(), scheduleDailyReminder(), cancelDailyReminder(), showNow()), NoopReminderService.

4.7 lib/server/ — сервер API

Сервер — отдельная программа, в приложение на телефоне он не входит. Запускается из bin/marshrut_server.dart, в тестах работает прямо в памяти.

Таблица 8 — Сервер API (папка lib/server/)

Файл	За что отвечает
api_server.dart	Все 39 адресов API и их обработчики. Каждый запрос проходит цепочку: журнал (строка «время, метод, адрес → код, мс» в окне сервера) → перехват ошибок (любая ошибка превращается в JSON {"error": …} с кодом, неожиданные — в 500 без подробностей) → маршрутизатор shelf_router → обработчик. Обработчик определяет пользователя по токену (401, если токена нет или он устарел), проверяет тело запроса теми же Validators, что и формы приложения (400), проверяет, что запись принадлежит пользователю (404 для чужих), и вызывает репозиторий drift. Токен — 32 случайных байта, в базе хранится только его хэш SHA-256, срок 30 дней. Главное: ApiServer (handler, _buildRouter(), _logRequests(), _handleErrors(), issueToken(), _userId(), _ownPlan(), _ownNote(), _register(), _login(), _createPlan(), _createNote() и др.), HttpError.

4.8 `lib/presentation/providers/` — связь экранов с данными

Провайдер Riverpod — это «поставщик» объекта или данных: экран просит у него, например, активный план и автоматически перерисовывается, когда план меняется на сервере.

Таблица 9 — Провайдеры и контроллеры состояния (папка lib/presentation/providers/)

Файл	За что отвечает
`providers.dart`	Все провайдеры Riverpod в одном месте, граф зависимостей «клиент API → репозитории API → сервисы → данные для экранов». В начале файла памятка по видам провайдеров; настройки и HTTP-клиент подставляют main() и тесты через overrides. Ответ сервера 401 переводит на экран входа (onSessionExpired). Адрес сервера хранит ServerUrlController. Главное: noAutoRetry(), sharedPreferencesProvider, httpClientProvider, apiClientProvider, authRepositoryProvider, planServiceProvider, authControllerProvider, currentUserIdProvider, directionTopicsProvider, activePlanProvider, planProgressProvider, notesProvider, ThemeModeController / themeModeProvider, ServerUrlController / serverUrlProvider.
`auth_controller.dart`	Состояние входа (неизвестно / вошёл / не вошёл), его слушает роутер. Выполняет автовход (с проверкой токена на сервере), вход, регистрацию, выход (с удалением токена на сервере), реакцию на истёкший токен, удаление аккаунта, правку профиля и синхронизацию ежедневного напоминания. Главное: AuthStatus, AuthState, AuthController (restoreSession(), login(), logout(), onSessionExpired(), setReminder(), _syncReminder()).
`onboarding_controller.dart`	Черновик нового плана, который заполняется на шагах онбординга: направление, ответы теста, расписание. По ответам теста определяются уже известные темы. Главное: OnboardingDraft (knownTopicIds, copyWith()), OnboardingController (selectDirection(), answer(), skipTest(), updateSettings(), reset()), onboardingProvider, defaultDeadline().

4.9 `lib/presentation/widgets/` — общие виджеты

Таблица 10 — Общие виджеты (папка lib/presentation/widgets/)

Файл	За что отвечает
`common.dart`	Общие виджеты и функции интерфейса для всех экранов: ограничение ширины на широких экранах, логотип, кольцо прогресса, заглушки пустого состояния, ошибки и загрузки, заголовок раздела, аватар, всплывающее сообщение, выполнение действия с показом ошибки и диалог подтверждения. runAction() выполняет действие (отметить занятие, удалить заметку) и, если сервер вернул ошибку или недоступен, показывает красное сообщение внизу экрана. Главное: ContentFrame, AppLogo, ProgressRing, EmptyState, ErrorView, LoadingView, SectionTitle, UserAvatar, showSnack(), runAction(), confirm(), directionIcon(), kWideLayoutBreakpoint.
`server_ address.dart`	Кнопка «Сервер: адрес» под формой входа и окно смены адреса. Перед сохранением приложение проверяет связь (`GET /api/health`) и что ответил именно сервер «Маршрута», поэтому неверный адрес не сохранится. Главное: ServerAddressButton, ServerAddressDialog, _ ServerAddressDialogState._save().
`session_tile. dart`	Карточка одного учебного занятия в списке: чекбокс «выполнено», цветная полоска типа (изучение, повторение, пропущено), название и подпись «длительность · тип · дата». Используется на главной и на экране плана. Главное: SessionTile, build().

4.10 `lib/presentation/screens/` — экраны

Каждый раздел приложения — отдельная подпапка; пути в таблицах даны от `lib/presentation/screens/`. Экран получает данные из провайдеров, а действия пользователя передаёт в контроллеры и сервисы.

Таблица 11 — Экраны запуска, вкладок и входа

Файл	За что отвечает
splash_screen.dart	Экран-заставка при запуске: анимация логотипа, заранее загружает каталог тем с сервера (ждёт не больше 3 секунд, чтобы при выключенном сервере не зависнуть) и проверяет «Запомнить меня», после чего роутер сам переводит на главную или на вход. Главное: SplashScreen, _SplashScreenState._start(), _warmUp().
shell/main_shell.dart	Каркас основной части приложения с пятью вкладками (Главная, План, Темы, Статистика, Профиль). На телефоне панель вкладок внизу, на широком экране сбоку. Главное: MainShell, destinations, _select(), build().
auth/login_screen.dart	Экран входа: поля email и пароль с проверкой, флажок «Запомнить меня», вывод ошибки входа (в том числе «Нет связи с сервером»), ссылка на регистрацию и кнопка адреса сервера. Сам вход выполняет AuthController. Главное: LoginScreen, _LoginScreenState._submit(), build().
auth/register_screen.dart	Экран регистрации: имя, email, пароль и его повтор с проверкой; ошибка «email занят» выводится под полем. После успеха открывается экран входа с уже введённым email. Главное: RegisterScreen, _RegisterScreenState._submit(), build().

Таблица 12 — Экраны создания плана (онбординг)

Файл	За что отвечает
onboarding/onboarding_steps.dart	Индикатор шагов мастера создания плана: три полоски «Направление / Тест / Расписание», пройденные шаги подсвечены оранжевым. Главное: OnboardingSteps (поле current, labels).
onboarding/direction_screen.dart	Шаг 1 онбординга (маршрут /onboarding): список направлений из каталога. По нажатию направление запоминается в черновике плана и открывается тест. Главное: DirectionScreen, directionsProvider, onboardingProvider.notifier.selectDirection().
onboarding/diagnostic_screen.dart	Шаг 2 (маршрут /onboarding/test): диагностический тест, по вопросу на ключевую тему, есть вариант «Не знаю». В конце экран результата со списком тем, которые уже известны и не войдут в план. Тест можно пропустить. Главное: DiagnosticScreen, _DiagnosticScreenState (_index, _finished), _OptionTile, answer(), skipTest(), knownTopicIds.
onboarding/schedule_screen.dart	Шаг 3 (маршрут /onboarding/schedule): часы в неделю, учебные дни, дата начала и дедлайн. Здесь же общая форма расписания, которую использует и экран настроек плана. Главное: ScheduleScreen, ScheduleForm (settings, onChanged, showStartDate, pick()).
onboarding/preview_screen.dart	Предпросмотр плана перед сохранением (маршрут /onboarding/preview). Показывает сводку, предупреждение о дедлайне с кнопкой «Установить N ч/нед» и порядок тем; кнопка «Начать обучение» сохраняет план. Главное: PreviewScreen, _PreviewScreenState._save(), PlanService.preview()/createPlan(), CycleDetectedException, _PreviewBody, _Stat.

Таблица 13 — Главная, план и занятия

Файл	За что отвечает
home/home_screen.dart	Главная вкладка (/home). Показывает приветствие, синюю карточку прогресса активного плана (кольцо, часы, серия дней, дедлайн), баннер о пропущенных занятиях с кнопкой «Перестроить» и занятия на сегодня. Если плана нет, предлагает его создать. Главное: HomeScreen, _HomeBody, _HomeBody._streak(), _ProgressCard, _MissedBanner, _MissedBannerState._replan().
plan/plan_screen.dart	Вкладка «План» (маршрут /plan): занятия активного плана по дням, полоса прогресса, поиск по темам и чипы-фильтры. Главное: enum SessionFilter (matches()), PlanScreen, _PlanScreenState._sessions(), PlanProgress.fromSessions(), SessionTile.
plan/session_screen.dart	Карточка занятия (маршрут /session/:id): тема, дата, длительность, совет по повторению, описание темы (TopicDetails), отметка «выполнено» и переход к Pomodoro. Здесь же объявлен общий провайдер темы по id, его использует и TopicScreen. Главное: _sessionProvider, SessionScreen, _SessionBody, topicByIdProvider.
plan/plan_settings_screen.dart	Настройки активного плана (маршрут /plan/settings): меняет расписание и перестраивает невыполненные занятия с сегодняшнего дня. Главное: PlanSettingsScreen, _PlanSettingsScreenState (_settings, _saving), ScheduleForm, PlanService.updateSettings().
plan/plans_screen.dart	«Мои планы» (маршрут /plans): все планы пользователя карточками с прогрессом. Через меню карточки план можно сделать активным, переименовать, перестроить или удалить; кнопка «Новый план» запускает онбординг. Главное: PlansScreen, _PlanCard (_activate(), _onAction(), _askTitle()), enum _PlanAction, _RenameDialog.

Таблица 14 — Экраны тем и заметок

Файл	**За что отвечает**
`topics/topics_screen.dart`	Вкладка «Темы» (/topics). Показывает каталог выбранного направления вместе со своими темами, поиск, фильтры «Все / Избранное / Мои / категории» и кнопку «Своя тема». Выбранное направление хранится в отдельном провайдере. Главное: SelectedDirection, selectedDirectionProvider, TopicsScreen, _TopicsScreenState._list(), _chipButton(), _DirectionPicker, _TopicCard.
`topics/topic_screen.dart`	Экран одной темы (/topic/:id): название, категория и TopicDetails. У своей темы здесь же можно изменить её, удалить или добавить в активный план (с проверкой направления). Главное: TopicScreen, TopicScreen._delete(), _AddToPlanButton.
`topics/topic_details.dart`	Общий блок подробностей темы: описание, оценка времени, материалы с копированием ссылки, заметки по теме, кнопка «Добавить заметку» и звёздочка избранного. Его вставляют экран темы и экран занятия. Главное: TopicDetails, TopicDetails.build().
`topics/custom_topic_screen.dart`	Форма создания и изменения своей темы (/topics/custom/new и /topics/custom/:id). В ней название, описание, часы, категория и выбор тем каталога, после которых изучать эту тему (зависимости для топологической сортировки). Главное: CustomTopicScreen, _CustomTopicScreenState._load(), _save().
`notes/notes_screen.dart`	Экран «Мои заметки» (/notes, открывается из профиля): все заметки пользователя с поиском по заголовку, тексту и теме, плюс кнопка «+» для новой заметки. Главное: NotesScreen, _NotesScreenState.build().
`notes/note_edit_screen.dart`	Редактор заметки (/notes/edit): создание, изменение и удаление (с подтверждением). Заметку можно привязать к теме через параметры адреса topicId и topicTitle. Главное: NoteEditScreen, _NoteEditScreenState._load(), _save(), _delete().

Таблица 15 — Экраны статистики, профиля и Pomodoro

Файл	**За что отвечает**
`stats/stats_screen.dart`	Вкладка «Статистика» (маршрут /stats): три плитки с итогами (часы учёбы, занятия, помидоры), столбчатая диаграмма часов за последние 6 недель, круговая диаграмма по категориям и история последних 30 записей; графики рисует библиотека fl_chart. Главное: StatsScreen, _history(), _Tile, _WeeklyChart, _CategoryChart.
`profile/profile_screen.dart`	Вкладка «Профиль» (/profile): данные пользователя, три показателя (планы, занятия, часы), переходы к редактированию профиля, паролю, заметкам и планам, выбор темы оформления, ежедневное напоминание, строка «Сервер API» с текущим адресом, выход и удаление аккаунта. Главное: ProfileScreen, _setReminder(), _time(), _Metric, _SegmentLabel.
`profile/edit_profile_screen.dart`	Экран «Редактировать профиль» (/profile/edit): смена имени и email, выбор цвета аватара из avatarPalette с превью, сохранение через AuthController.updateProfile. Главное: EditProfileScreen, _EditProfileScreenState._save().
`profile/change_password_screen.dart`	Экран «Сменить пароль» (/profile/password): форма с текущим паролем, новым паролем и его повтором, проверка через Validators, смена пароля через AuthController.changePassword. Главное: ChangePasswordScreen, _ChangePasswordScreenState._save().
`profile/delete_account_dialog.dart`	Диалог удаления аккаунта с повторным вводом пароля: удаляет аккаунт через AuthController.deleteAccount и возвращает true при успехе. Главное: showDeleteAccountDialog(), _DeleteAccountDialog, _DeleteAccountDialogState._delete().
`pomodoro/pomodoro_screen.dart`	Экран таймера Pomodoro (/pomodoro, тема передаётся параметром topic): режимы 25/5 и 50/10, оставшееся время считается от момента окончания интервала, завершённый фокус отправляется на сервер (`POST /api/pomodoros`) через pomodoroRepositoryProvider, о конце интервала приходит уведомление, есть счётчик помидоров за сегодня. Главное: PomodoroMode, PomodoroScreen, _PomodoroScreenState (_start(), _pause(), _reset(), _tick(), _complete()), _TimerPainter.

5. Тесты

Тесты запускаются командой `flutter test` (137 тестов), интеграционный — `flutter test integration_test` (на компьютере с Linux или Windows). Все они прогоняются в GitHub Actions при каждой отправке кода. В виджет-тестах приложение работает с настоящим сервером API, у которого база в памяти, а запросы передаются ему напрямую, без сети.

Таблица 16 — Файлы тестов

Файл	**Что проверяет**
test/core/ formatters_test.dart	Склонение слов после чисел («1 час», «2 часа», «5 часов»), вывод часов и минут
test/core/password_ hasher_test.dart	PBKDF2-HMAC-SHA256 по эталонным значениям из стандарта, кириллические пароли, соль, старые хэши SHA-256
test/core/router_ test.dart	Правило доступа: пока сессия проверяется — только Splash, без входа — только вход и регистрация, после входа — главная
test/core/ validators_test.dart	Проверка полей форм: email, пароль, имя, повтор пароля, часы в неделю
test/data/catalog_ test.dart	Каталог: все 4 направления читаются без ошибок, повреждённый JSON даёт понятное сообщение, каталог кэшируется
test/data/api_ client_test.dart	Клиент API: адрес сервера, токен и JSON в запросе, ответ 204, кириллица, проверка связи; ошибка сервера с текстом, 500, ответ не JSON, 401, нет связи, таймаут; обновление потоков после изменений и повтор после ошибки
test/data/ repositories_test. dart	Репозитории сервера на базе в памяти: регистрация хранит хэш, уникальность email, вход, смена пароля, удаление аккаунта, планы и каскадное удаление, перепланирование, свои темы, избранное, заметки
test/server/api_ server_test.dart	Сервер API: служебные адреса, регистрация и вход, токены (срок 30 дней, в базе только хэш), неверные данные (400), без токена (401), чужие данные (404), сбой базы (500 без подробностей и запись в журнал), CRUD заметок и своих тем, избранное, планы через клиент API, Pomodoro
test/domain/ knowledge_estimator_ test.dart	Тест знаний: верный ответ отмечает тему и её основы, неверный исключает тему
test/domain/ topological_sort_ test.dart	Алгоритм Кана: каждая тема после своих зависимостей, обнаружение цикла, крайние случаи
test/domain/ scheduler_test.dart	Раскладка: дневной лимит, только выбранные дни, деление тем на части не короче 30 минут, повторения через 1, 3 и 7 дней
test/domain/plan_ generator_test.dart	Генератор: исключение известных тем, дата окончания, подбор часов под дедлайн
test/domain/ replanner_test.dart	Перепланирование: пропуски переносятся, изученное не повторяется, проверка дедлайна
test/presentation/ auth_screens_test. dart	Экраны входа и регистрации: ошибки под полями, занятый email, автовход, смена адреса сервера с проверкой связи, понятная ошибка без связи с сервером
test/presentation/ features_test.dart	Заметки, свои темы, «Мои планы», статистика, Pomodoro; сервер выключился — сообщение об ошибке, включился — действие проходит
test/presentation/ profile_test.dart	Напоминание, недоступность напоминания на компьютере, удаление аккаунта
test/presentation/ app_flow_test.dart	Полный сценарий: регистрация, вход, тест, расписание, план, отметка занятия, вкладки, выход
test/presentation/ adaptive_test.dart	Все экраны без переполнения на телефоне и в окне компьютера

Файл	Что проверяет
test/helpers/test_app.dart	Не тест, а помощник: запускает приложение с сервером API в памяти (inMemoryClient) и фиксированной датой
integration_test/app_test.dart	Настоящее приложение и настоящий HTTP-сервер API на 127.0.0.1: Splash → регистрация → вход → создание плана → профиль → выход

6. Остальные папки

Таблица 17 — Вспомогательные программы, данные и документы

Путь	За что отвечает
assets/catalog/*.json	Каталог: направление, темы (название, описание, категория, уровень, длительность, материалы), зависимости между темами и вопросы теста. Читает сервер API при запуске (bin/marshrut_server.dart) и отдаёт приложению по адресу /api/directions
tool/screenshots/screenshots_test.dart	Снимает скриншоты всех экранов для курсовой (docs/coursework/screenshots/)
tool/screenshots/qa_matrix_test.dart	Матрица проверки: каждый экран в 7 вариантах (светлая и тёмная тема, маленький телефон, крупный шрифт, горизонтально, окно компьютера)
tool/screenshots/screenshot_support.dart	Общие функции для снимков: тестовые данные, шрифты, сохранение PNG
tool/print_schedule_example.dart	Печатает пример расписания для таблицы 2.5 курсовой
tool/server/Запустить сервер.bat	Файл запуска сервера для Windows: кладётся рядом с программой-сервером в сборке GitHub Actions, запускает её на порту 8080 с базой marshrut_server.sqlite в той же папке
tool/icons/generate_icons.py	Рисует иконку приложения для всех платформ
tool/coursework/build_coursework.py	Собирает курсовую в DOCX и PDF из Markdown по ГОСТ: титульный лист, содержание, схемы, таблицы
tool/coursework/build_criteria_report.py	Собирает отчёт по критериям оценивания
tool/coursework/build_guide.py	Собирает памятки к защите, в том числе этот путеводитель
docs/coursework/	Текст курсовой по главам, тестирование, журнал ошибок, схемы, скриншоты, готовые DOCX и PDF
docs/concept/	Концепция первого семестра: слайды, пользовательский путь, макеты экранов
docs/criteria_semester1.md, docs/criteria_semester2.md	Таблицы «критерий → где реализовано → как показать»
docs/PROGRESS.md	Чек-лист выполнения по этапам
docs/Сервер_API.md	Как скачать и запустить сервер, подключить телефон, показать работу через API на защите; таблицы кодов ответов и всех адресов API
docs/Установка_на_iPhone.md	Как установить приложение на iPhone с компьютера Windows программой Sideloadly и подключить его к серверу

Путь	За что отвечает
android/app/src/main/AndroidManifest.xml	Название «Маршрут», иконка, разрешение на сеть и на запросы по HTTP к серверу в локальной сети, приёмники плагина уведомлений, которые восстанавливают напоминание после перезагрузки телефона
ios/Runner/Info.plist	Название «Маршрут» на iPhone, разрешение на запросы по HTTP и пояснение к запросу доступа к локальной сети
android/app/build.gradle.kts	Настройки сборки APK, идентификатор приложения com.barin.marshrut
windows/runner/	Окно программы для Windows: заголовок «Маршрут», начальный размер 420×880, иконка

7. Где искать, если спросят про…

Таблица 18 — Функция приложения и файлы, где она реализована

Функция	Где в коде
Регистрация и вход	screens/auth/, providers/auth_controller.dart, data/api/api_auth_repository.dart, на сервере data/repositories/auth_repository_impl.dart
Сервер API: адреса, токен, коды ответов	server/api_server.dart, запуск bin/marshrut_server.dart
Запросы к серверу и обработка ошибок	data/api/api_client.dart, runAction() в widgets/common.dart, ErrorView
Адрес сервера	widgets/server_address.dart, ServerUrlController в providers/providers.dart, data/repositories/session_storage_impl.dart
Хранение паролей	core/utils/password_hasher.dart
«Запомнить меня» и токен входа	data/repositories/session_storage_impl.dart, providers/auth_controller.dart, issueToken() и _userId() в server/api_server.dart
Закрытые экраны и переходы	core/router/app_router.dart (authRedirect)
Тест знаний	screens/onboarding/diagnostic_screen.dart, domain/planner/knowledge_estimator.dart
Порядок тем	domain/planner/topological_sort.dart (алгоритм Кана)
Раскладка по дням и повторения	domain/planner/scheduler.dart
Проверка дедлайна	domain/planner/plan_generator.dart (requiredHoursPerWeek)
Перепланирование	domain/planner/replanner.dart, domain/usecases/plan_service.dart
База данных и таблицы (на сервере)	data/db/app_database.dart
CRUD: заметки	screens/notes/, data/api/api_note_repository.dart, на сервере data/repositories/note_repository_impl.dart
CRUD: свои темы и избранное	screens/topics/custom_topic_screen.dart, data/api/api_topic_repository.dart, на сервере data/repositories/topic_repository_impl.dart
CRUD: планы	screens/plan/plans_screen.dart, data/api/api_plan_repository.dart, на сервере data/repositories/plan_repository_impl.dart

Функция	**Где в коде**
Поиск и фильтры	`screens/plan/plan_screen.dart`, `screens/topics/topics_screen.dart`, `screens/notes/notes_screen.dart`
Статистика и графики	`screens/stats/stats_screen.dart` (пакет fl_chart)
Уведомления	`data/services/reminder_service_impl.dart`
Pomodoro	`screens/pomodoro/pomodoro_screen.dart`, `ApiPomodoroRepository` в `data/api/api_note_repository.dart`
Тема оформления	`core/theme/app_theme.dart`, `ThemeModeController` в `providers/providers.dart`
Адаптивность (телефон и окно Windows)	`widgets/common.dart` (`ContentFrame`), `screens/shell/main_shell.dart`
Удаление аккаунта	`screens/profile/delete_account_dialog.dart`, `DELETE /api/me` (`_deleteAccount` в `server/api_server.dart`), `AuthRepositoryImpl.deleteAccount()`